Lab 5.1 - Storage Pools & Additional Disks

KVM Virtualisation on Oracle Linux - Day 2, Module 5

Overview

In this lab you will build a new storage pool on a dedicated secondary disk, create a volume inside it, and attach that volume to ol-lab-01 as a second disk while it's running - then go inside the guest and make that disk actually usable. You'll finish by comparing qcow2 and raw volume creation directly, to see the format difference from Module 5's guide in practice rather than just in theory.

This lab assumes ol-lab-01 exists, is running, and is connected to the bridged network from Lab 4.1.

> Note: Your OCI lab instance has a dedicated secondary block volume attached via Terraform, separate from the boot volume everything else so far has been sharing. It was partitioned, formatted, and mounted at /mnt/data as a manual exercise during the Day 1 Linux training - Terraform only attaches the raw disk, it does not prepare it. This lab uses that already-prepared volume rather than /var/lib/libvirt/images on the boot volume, both because it's what the environment was built for and because the boot volume does not have generous free space to spare by this point in the day.

Learning Objective

By the end of this lab, you will be able to:

- Confirm and use a pre-attached secondary block device as dedicated storage
- Create and start a directory-based storage pool on dedicated storage
- Create a qcow2 volume inside a pool
- Attach a volume to a running VM, live and persistently
- Partition, format, and mount a new disk inside a guest
- Compare qcow2 and raw volumes directly on creation time and file size

Step by Step Guide

Prerequisite: Confirm System-Level virsh Access

Every command in this lab needs to reach the same libvirt connection ol-lab-01 lives on - not a private per-user session that would silently create the pool somewhere disconnected from the VM.

1. Confirm ol-lab-01 is visible and you're talking to the system connection:

   groups

   sudo virsh list --all

   Expected result: ol-lab-01 listed. Commands throughout this lab use sudo consistently to guarantee they land on qemu:///system, regardless of your session's group configuration.

   Note: If ol-lab-01 does not appear even with sudo, stop here and re-check Lab 3.1 before continuing - nothing in this lab will work correctly against the wrong VM or the wrong connection.

Step 1: Confirm the Pre-Provisioned Data Volume

This lab's secondary disk is already partitioned, formatted, and mounted, prepared during the Day 1 Linux lab - there is nothing to build here, only to confirm. Do not run any partitioning or formatting commands against this device; it already holds a filesystem.

2. List all block devices and identify the boot disk versus the secondary data disk:

   lsblk

   Note: The boot disk is the one with existing partitions, one of which is mounted at / - do not touch that one. The secondary data disk shows as a single ~50G device, commonly /dev/sdb on OCI instances, with a single partition already mounted - confirm the exact device name and mount point against your own output rather than assuming.

3. Confirm the data disk's partition and mount point:

   sudo lsblk /dev/sdb

   Expected result: sdb1 listed with MOUNTPOINTS showing /mnt/data (or the equivalent path shown in your own output) - the disk is already partitioned and mounted, prepared this way during the Day 1 Linux lab rather than something to build fresh here.

4. Confirm the filesystem type and available space:

   df -hT /mnt/data

   Expected result: The mount shows close to the full ~50G available - this is the headroom the rest of this lab will use, well clear of the boot volume's tighter free space.

   Note: If your output shows a different mount point than /mnt/data, use that path consistently in place of /mnt/data throughout the rest of this lab.

Step 2: Create the Storage Pool on the Dedicated Disk

Build a new directory-based pool named labpool, pointed at the mounted data disk rather than the boot volume.

5. Define the pool against the mount point from Step 1:

```
sudo virsh pool-define-as labpool dir --target /mnt/data
```

Expected result: Pool labpool defined.

6. Build, start, and autostart it:

```
sudo virsh pool-build labpool
sudo virsh pool-start labpool
sudo virsh pool-autostart labpool
```

Expected result: Each command confirms success in turn, with no errors.

7. Confirm it's active alongside the default pool, and check its reported capacity:

```
sudo virsh pool-list --all --details
```

Expected result: Both default and labpool listed as active, autostart yes - labpool shows close to 50 GiB capacity, confirming it's genuinely backed by the dedicated disk and not the boot volume.

Step 3: Create a 10 GB qcow2 Volume

8. Create the volume inside labpool:

```
sudo virsh vol-create-as labpool data-disk.qcow2 10G --format qcow2
```

Expected result: Vol data-disk.qcow2 created.

9. Confirm it exists and check its actual size on disk:

```
sudo virsh vol-list labpool
sudo qemu-img info /mnt/data/data-disk.qcow2
```

Expected result: virtual size shows 10 GiB, but disk size (the actual space used on the host) is only a few hundred KB - confirmation that qcow2 has not pre-allocated the full 10 GB.

Step 4: Attach the Volume to ol-lab-01

Attach the new volume as a second disk while the VM keeps running.

10. Attach the disk, live and persistently:

```
sudo virsh attach-disk ol-lab-01 \
```

```
  /mnt/data/data-disk.qcow2 \
  vdb --subdriver qcow2 --targetbus virtio --live --config
```

Expected result: Disk attached successfully.

11. Confirm it's visible from the host side:

```
sudo virsh domblklist ol-lab-01
```

Expected result: Two entries listed: vda (the VM's original disk) and vdb (the new one, pointing at data-disk.qcow2).

Step 5: Partition, Format, and Mount the New Disk Inside the Guest

The disk is attached but not yet usable - like a new physical drive, it needs a partition table and filesystem before the guest can store anything on it.

12. Find the VM's address and connect to it - no graphical console needed, consistent with how ol-lab-01 has been accessed since Lab 3.1:

```
sudo virsh domifaddr ol-lab-01
ssh root@<ol-lab-01's address>
```

13. Confirm the new disk is visible inside the guest:

```
lsblk
```

Expected result: A new 10G device named vdb listed, with no partitions yet.

14. Create a partition on it:

```
sudo parted /dev/vdb --script mklabel gpt mkpart primary ext4 0% 100%
```

15. Format the new partition:

```
sudo mkfs.ext4 /dev/vdb1
```

Expected result: mke2fs output confirming the filesystem was created.

16. Create a mount point and mount it:

```
sudo mkdir /data
sudo mount /dev/vdb1 /data
```

17. Confirm it's mounted and usable:

```
df -h /data
sudo touch /data/test-file
ls -l /data
```

Expected result: /data shows roughly 10G available, and test-file is created successfully - the disk is fully usable.

Note: This mount will not survive a reboot unless it's added to /etc/fstab. That's deliberate for this lab - the focus here is confirming the disk works, not building a production-ready fstab entry.

Step 6: Compare qcow2 and raw Volume Creation

Back on the host, create a second volume of the same nominal size in raw format, and compare the two directly. Because labpool now lives on a mostly-empty 50 GB disk, this comparison can run without any risk of the capacity concerns seen elsewhere today.

18. Time the creation of a 10 GB qcow2 volume and a 10 GB raw volume:

    time sudo virsh vol-create-as labpool compare-test.qcow2 10G --format qcow2

    time sudo virsh vol-create-as labpool compare-test.raw 10G --format raw

    Expected result: Both commands report a real time - typically very similar to each other, since neither format pre-writes 10 GB of data at creation time by default.

19. Compare their actual size on disk:

    sudo qemu-img info /mnt/data/compare-test.qcow2

    sudo qemu-img info /mnt/data/compare-test.raw

    Expected result: Both report a virtual size of 10 GiB. The disk size (actual space consumed) is small and near-identical for both immediately after creation, since neither has been written to yet - the real difference between the formats shows up over time as the VM writes data, not at creation.

20. Clean up the comparison volumes, since they were created for this exercise only:

    sudo virsh vol-delete compare-test.qcow2 --pool labpool

    sudo virsh vol-delete compare-test.raw --pool labpool

Outcomes

By the end of this lab, ol-lab-01 has a second, fully usable disk - partitioned, formatted, and mounted at /data - created from a storage pool built on dedicated secondary storage rather than the shared boot volume. You have confirmed and put a pre-attached block device to use, attached a disk live to a

running VM without any downtime, and directly compared qcow2 and raw volume creation rather than relying on the format guide alone. This second disk and pool carry forward into Module 6, where you will take a snapshot of the VM's state before deliberately introducing a fault.